

An Architectural Digital Twin for Pre-Deployment Verification and CI/CD Gating in Air Traffic Management Middleware Systems (Industry Track)

Aleksander Slominski
IBM Research
Yorktown Heights, NY, USA
aslom@us.ibm.com

Eduard Marin
Telefonica
Madrid, Spain
eduard.marin@telefonica.com

Javier Picorel
Huawei Research
Munich, Germany
javier.picorel@huawei.com

Onur Alp Yigit
System as a Graph Research Team
Ankara, Turkey
onuralp@systemasagraph.org

Abstract

Modern mission-critical Air Traffic Management (ATM) systems rely on complex distributed middleware infrastructures deployed across multi-tier controller workstations, surveillance radar processing gateways, and flight data processing systems. In continuous delivery pipelines, software updates undergo frequent releases across varying operational scales—from terminal maneuvering areas (TMA) to multi-center en-route sectors. However, non-local architectural misconfigurations, such as Quality-of-Service (QoS) parameter incompatibilities, conflicting hardware processor core allocations, memory parameter misalignments, and circular package dependencies, frequently pass unit and integration testing. These defects manifest only as costly post-deployment failures or runtime degradations in target operational airspace environments.

This paper presents **System as a Graph (SaaG)**, an architectural digital twin for pre-deployment verification and continuous deployment gating in ATM middleware systems. The twin is *static*: a typed multigraph ($G = (V, E)$) reconstructed fresh from candidate software versions, avoiding the complexity of live-synchronized runtime models. We contribute three main results: (1) an industrial scale ATM digital twin graph schema and requirements baseline specifying 5 entity classes, 6 structural relations, and 6 logical dependency rules derived from physical pub/sub linkage; (2) a CI/CD pipeline gate combining an implemented severity-based exit-code gate with a specified 4-heading installation suitability scoring model; and (3) performance benchmarks across 5 operational profiles (29 to 444 components) demonstrating sub-second verification latencies (0.01 s to 1.01 s). Furthermore, we document an industrial gap analysis showing that 4 of 7 specified verification checks are blocked on configuration data acquisition rather than graph analysis techniques, and detail a structured experimental methodology to evaluate defect detection effectiveness, detector precision scaling, and GNN-based criticality ranking upon completion of full SaaG-D system deployment.

CCS Concepts

• **Computer systems organization** → **Real-time systems**; • **Software and its engineering** → **Software maintenance tools**.

Keywords

Air Traffic Management (ATM), Architectural Digital Twin, Middleware Verification, Pub/Sub QoS Contracts, CI/CD Gating, CPU Core Pinning, Static Rule Auditing

ACM Reference Format:

Aleksander Slominski, Javier Picorel, Eduard Marin, and Onur Alp Yigit. 2026. An Architectural Digital Twin for Pre-Deployment Verification and CI/CD Gating in Air Traffic Management Middleware Systems (Industry Track). In *Proceedings of ACM International Middleware Conference (Middleware '26)*. ACM, New York, NY, USA, 6 pages.

1 Introduction & Industrial ATM Problem Statement

Continuous Integration and Continuous Delivery (CI/CD) pipelines have streamlined building and testing software artifacts. However, in mission-critical Air Traffic Management (ATM) environments governed by publish/subscribe (pub/sub) middleware, verifying an assembled distributed system prior to installation remains a major operational challenge.

1.1 The Pre-Deployment Verification Gap in ATM Systems

Governed by the ICAO Global ATM Operational Concept [1], ATM systems execute across multi-core processing servers, radar processing gateways, and controller workstations. Software units exchange high-frequency telemetry and flight control data via pub/sub channels.

When a candidate software build is proposed for release into an operational sector, conventional CI/CD pipelines evaluate unit test suites in isolation. Consequently, systemic architectural misconfigurations pass unnoticed:

- **Hardware Core Contention:** Latency-sensitive surveillance processes pinned to overlapping CPU cores, or host node allocations exceeding physical CPU core capacities on multi-core servers.

- **Middleware QoS Contract Incompatibilities:** Mismatched Quality-of-Service (QoS) parameters on critical safety channels. For instance, a subscriber expecting TRANSIENT_LOCAL durability for emergency clearance commands bound to a publisher configured for VOLATILE delivery; in pub/sub middleware, mismatched QoS contracts cause silent transport disconnections.
- **Silent Topic Disconnections:** Software units publishing to or consuming from topics with schema definitions that diverge across releases, or refactored topics left with no active subscribers.
- **Circular Package Dependencies:** Transitive dependency cycles between flight plan processors and 4D trajectory predictors that manifest as initialization deadlocks during sector start-up.

Building testbeds for every candidate build across facilities is prohibitively slow and expensive. Furthermore, staging networks fail to capture full operational loads, leaving architectural configuration defects to be discovered during live operations.

1.2 The SaaG Approach & Contributions

To bridge this gap, we present **System as a Graph (SaaG)**, an architectural digital twin framework for pre-deployment verification in ATM middleware systems. SaaG constructs a static, multi-layered digital model of the candidate system topology without executing application binaries. Operating directly within CI/CD workflows, SaaG statically audits hardware core bindings, topic QoS parameters, and package dependencies before software installation.

This paper makes the following contributions:

- (1) **ATM Digital Twin Schema & Requirements Baseline (§2):** A formal graph representation ($G = (V, E, \tau_V, \tau_E, w)$) mapping 5 entity classes and 6 relation types, supported by a verifiable requirements baseline developed with system architects.
- (2) **CI/CD Gating & Industrial Gap Analysis (§3, §4):** A two-stage deployment gate comprising an implemented severity exit-code gate (codes 0/1/2) and a specified 4-heading Installation Suitability Scoring model (S). We report an explicit gap analysis showing why 4 of 7 specified verification checks are blocked on input data acquisition.
- (3) **Multi-Scale Performance & Evaluation Plan (§5):** Empirical benchmarks across 5 scaling ATM deployment profiles (29 to 444 components) demonstrating sub-second audit latencies (0.01 s to 1.01 s), accompanied by a structured experimental methodology to evaluate defect detection, detector precision scaling, and criticality model accuracy upon completion of full system deployment.

2 System Overview & ATM Digital Twin Graph Model

2.1 Static Scoping & System Capabilities

“Digital twin” is scoped precisely: SaaG builds a **static twin**—a typed graph reconstructed fresh for each candidate build, rather than a continuously live-synchronized model. A complete digital twin

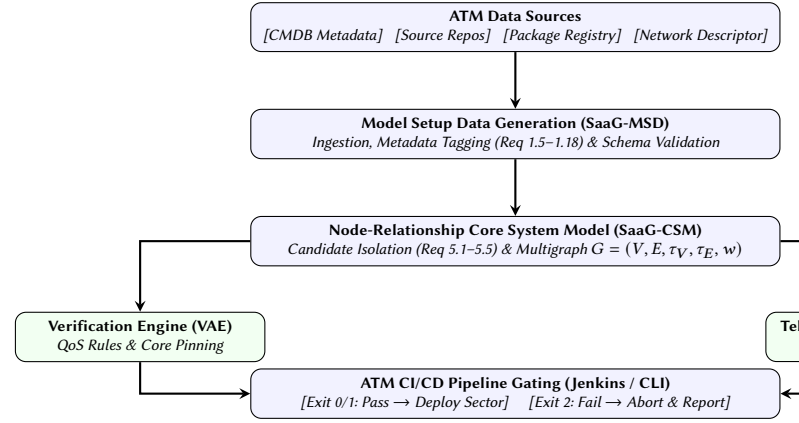


Figure 1: SaaG Architectural Digital Twin Pipeline Integration Overview for ATM Systems.

possesses a dynamic half (live telemetry overlay and design-versus-observed drift detection). In SaaG, this dynamic half is specified in the requirements baseline, but separated from the static verification engine reported here.

To enforce clarity, we distinguish **SaaG-D** (will be deployed baseline system specification) from **SaaG-P** (the open verification prototype).

2.2 Model Setup Data Generation (SaaG-MSD)

The **Model Setup Data Generation (SaaG-MSD)** component interfaces with four enterprise repositories: CMDB metadata, source control repositories, package registries, and network topology descriptors. Each acquired record is tagged with project, platform, sector ID, and candidate release version numbers.

2.3 Core System Model Formal Graph Definition (SaaG-CSM)

The **Core System Model (SaaG-CSM)** transforms ingested descriptors into a multi-attributed, weighted directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

2.3.1 ATM Vertex Entity Types (V). The vertex set $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}}$ classifies 5 entity types:

- **Applications (V_{app}):** Executable ATM software units.
- **Brokers (V_{broker}):** ATM message brokers.
- **Topics (V_{topic}):** Pub/sub channels.
- **Infrastructure Nodes (V_{node}):** Processing hardware hosts.
- **Libraries (V_{lib}):** Shared code modules.

2.3.2 Structural Edge Types ($E_{\text{structural}}$). Six structural edge types are imported: PUBLISHES_TO ($(V_{\text{app}} \cup V_{\text{lib}}) \times V_{\text{topic}}$), SUBSCRIBES_TO ($V_{\text{topic}} \times (V_{\text{app}} \cup V_{\text{lib}})$), ROUTES ($V_{\text{broker}} \times V_{\text{topic}}$), RUNS_ON ($(V_{\text{app}} \cup V_{\text{broker}}) \times V_{\text{node}}$), CONNECTS_TO ($V_{\text{node}} \times V_{\text{node}}$), and USES ($(V_{\text{app}} \cup V_{\text{lib}}) \times V_{\text{lib}}$).

2.3.3 Derived Dependency Projection Rules ($E_{dependency}$). Physical pub/sub data flows from publisher to subscriber. However, structural failure dependencies point in the opposite direction: a subscriber depends on the publisher supplying its topic data. SaaG derives DEPENDS_ON edges pointing **from dependent to dependency**:

Table 1: Derived Logical Dependency Edge Projection Rules.

Rule	Type	Derivation Pattern	Weight
1	app_to_app	App/Lib SUB $t \leftarrow$ PUB App/Lib	$\max_t w(t)$
2	app_to_broker	App/Lib PUB/SUB $t \leftarrow$ ROUTES Broker	$\max_t w(t)$
3	node_to_node	Lifted from app_to_app between hosted apps	Lifted $\max(w)$
4	node_to_broker	Lifted from app_to_broker for hosted apps	Lifted $\max(w)$
5	app_to_lib	App/Lib USES $\rightarrow$ Library	Inherits $w(\text{app})$
6	broker_to_broker	Colocation edge sharing physical Node	Inherits $w(\text{app})$

2.4 Intrinsic Pub/Sub QoS & Upward Weight Propagation

SaaG assigns criticality weights $w(v) \in [0, 1]$ across all entities using an Analytical Hierarchy Process (AHP) QoS formula [12]:

- (1) **Intrinsic Topic Weight Formula:** For each topic $t \in V_{\text{topic}}$,

$$w(t) = \max(0.01, \beta \cdot \text{QoS_score}(t) + (1 - \beta) \cdot \text{size_norm}(t)) \quad (2)$$

where $\beta = 0.85$, $\text{size_norm}(t) = \min\left(\frac{\log_2(1 + \text{size_kb})}{50}, 1.0\right)$, and:

$$\text{QoS_score}(t) = 0.30 \cdot \text{rel} + 0.40 \cdot \text{dur} + 0.30 \cdot \text{prio} \quad (3)$$

Reliability (RELIABLE=1.0, BEST_EFFORT=0.0), Durability (PERSISTENT=1.0, TRANSIENT_LOCAL=0.5, VOLATILE=0.0), and Transport Priority (CRITICAL=1.0, HIGH=0.66, LOW=0.0).

- (2) **Upward Weight Propagation:** Component weights propagate hierarchically:

$$w(a) = 0.80 \cdot \max_{t \in T(a)} w(t) + 0.20 \cdot \text{mean}_{t \in T(a)} w(t) \quad (4)$$

$$w(b) = 0.70 \cdot \max_{t \in T(b)} w(t) + 0.30 \cdot \text{mean}_{t \in T(b)} w(t) \quad (5)$$

$$w(l) = \min(1.0, \text{base_w} \cdot (1 + \gamma \cdot \log_2(1 + DG_{\text{in}}(l)))) \quad (6)$$

$$w(n) = \max_{v \text{ RUNS_ON } n} w(v) \quad (7)$$

where $\gamma = 0.15$ accounts for library fan-out amplification.

2.5 Multi-Session Process-Isolated Candidate Modeling

During concurrent builds, SaaG constructs an isolated candidate graph $G_{u'} = (V', E')$ by substituting candidate unit version u' into the target baseline inventory under a unique process identifier.

3 Verification Engine & Industrial Gap Analysis

The **Verification and Analysis (SaaG-VAE)** engine executes rule-based static audits, cascade simulations, and telemetry overlays on candidate digital twins $G_{u'}$.

3.1 Implemented Rule Audits

- (1) **Middleware Topic QoS Conformance:** For every topic $v_t \in V_{\text{topic}}$, publishers $P(v_t)$ and subscribers $S(v_t)$ are audited for QoS policy matching. If subscriber durability or reliability exceeds publisher guarantees, a critical incompatibility finding is logged.
- (2) **Processor Core Pinning & Contention:** For each host node $v_p \in V_{\text{node}}$ with physical core capacity $C(v_p)$, SaaG verifies $\sum_{u \in U(v_p)} |\text{Cores}(u)| \leq C(v_p)$ and enforces non-overlapping core sets ($\text{Cores}(u_i) \cap \text{Cores}(u_j) = \emptyset$) for latency-sensitive processes.
- (3) **Orphaned Topics & Schema Consistency:** Flags un-subscribed topics ($|S(v_t)| = 0$), un-published topics ($|P(v_t)| = 0$), and mismatched ASTERIX message category payloads across endpoints.
- (4) **Circular Dependencies:** Tarjan's Strongly Connected Components algorithm [11] detects circular package references.

3.2 Field Telemetry Overlay & Synthetic Cascade Simulation

- **Telemetry Overlay:** Operational telemetry from the Field Records Database is bound to graph nodes. SaaG detects architectural drift via $\text{Drift}_{\text{Undeclared}} = E_{\text{Observed}} \setminus E_{\text{Designed}}$.
- **Cascade Impact Simulation:** The Scenario Generator simulates node failures or message bursts along structural edges ($E_{\text{structural}}$), establishing a blast-radius reference oracle for scoring detector accuracy.

3.3 Specified vs. Implemented Gap Analysis

A key finding for industrial practitioners is the gap between specified verification requirements and actual prototype implementation. Table 2 enumerates 7 specified capabilities in SaaG-D that were not fully realized in SaaG-P:

Industrial Insight: 4 of the 7 unbuilt capabilities are blocked on **input data acquisition** (extracting configuration files from enterprise repositories into the graph pipeline), not on graph analysis algorithms.

4 CI/CD Pipeline Integration & Automated Gating

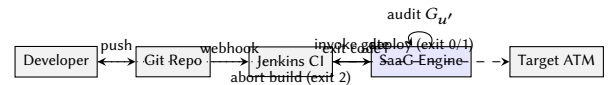


Figure 2: Sequence Diagram of SaaG Automated CI/CD Deployment Gating.

4.1 Implemented Severity Exit-Code Gate & Limitations

SaaG-P integrates into continuous delivery pipelines via a command-line interface. It evaluates candidate graph $G_{u'}$, runs detectors, and

Table 2: Specified vs. Implemented Gap Analysis in SaaG Verification Capability.

Verification Capability	Status in Prototype (SaaG-P)	Primary Operational Blocker
Endpoint QoS Conformance	Topic-level QoS anomaly check	Data Ingestion: Endpoint-level IDL descriptors missing
Payload Schema Matching	Topic string identity match	Data Ingestion: Message IDL field parser not integrated
Hardware Core Allocation	Core capacity attribute flag	Data Ingestion: OS core assignment files live outside build repos
OS Memory & Kernel Audit	Config descriptor requirement	Data Ingestion: Target OS parameters not exported to CMDB
Live Architectural Drift	Static vs observed graph diff	Data Ingestion: Operational field log telemetry store unbuilt
Scored Suitability Model	Severity exit-code gate (0/1/2)	Implementation: Scoring weights require empirical tuning
CMDB & Network Ingestion	JSON system descriptor files	Data Ingestion: Enterprise CMDB REST API integration pending

encodes its decision into standard process exit status:

$$\text{Exit Status} = \begin{cases} 2, & \text{if any CRITICAL or HIGH finding is detected} \\ 1, & \text{if MEDIUM or LOW findings are detected} \\ 0, & \text{if zero findings are detected} \end{cases} \quad (8)$$

Standard CI runners (e.g., Jenkins, GitHub Actions) treat exit code 2 as a build failure, aborting deployment without extra plugins.

4.1.1 Absolute vs. Delta-Aware Gating Limitation. The implemented gate evaluates the candidate system on an absolute basis rather than measuring deltas against the merge base. If a codebase carries pre-existing HIGH findings, every subsequent candidate build returns exit code 2. An absolute gate deployed without a waiver register would block all pipeline releases. In production pipelines, gating must be *delta-aware*—blocking candidate builds only when new CRITICAL or HIGH findings are introduced.

4.2 Specified Scored Installation Suitability Model

SaaG-D is specified to replace binary exit status with a quantitative **Installation Suitability Score** (S) evaluated across four headings: (1) Structural & Architectural Conformance (H_1), (2) Interface & Topic Conformance (H_2), (3) Dependency & Integration Conformance (H_3), and (4) Resource & Performance Sufficiency (H_4).

$$S = 100 - \sum_{k \in \text{Violations}} W(r_k) \cdot \text{Penalty}(S(r_k)) \quad (9)$$

$$\text{Decision}(u') = \begin{cases} \text{FAIL}, & \text{if } \exists k \text{ s.t. } S(r_k) = \text{Critical} \vee B(r_k) = 1 \\ \text{FAIL}, & \text{if } S < S_{\text{threshold}} \\ \text{PASS}, & \text{otherwise} \end{cases} \quad (10)$$

4.3 Actionable Findings & Evidence-Bearing Reports

SaaG outputs structured JSON finding reports containing the rule identifier, affected component, severity, root-cause description, and exact supporting evidence (e.g., conflicting core IDs or mismatched QoS enums), providing developers with diagnostic guidance.

5 Industrial ATM Evaluation & Multi-Scale Performance

5.1 Experimental Setup Across 5 ATM Scale Profiles

We evaluated SaaG performance across 5 scaling operational profiles modeled under the ICAO Global ATM Operational Concept framework:

- (1) **Scale Tiny (Approach Sector):** 10 Apps, 8 Topics, 2 Brokers, 6 Nodes, 3 Libraries (29 Components).
- (2) **Scale S (TMA Sector):** 26 Apps, 27 Topics, 5 Brokers, 8 Nodes, 8 Libraries (74 Components).
- (3) **Scale M (Regional ACC Center):** 52 Apps, 54 Topics, 10 Brokers, 16 Nodes, 16 Libraries (148 Components).
- (4) **Scale L (Multi-Sector ACC Cluster):** 104 Apps, 108 Topics, 20 Brokers, 32 Nodes, 32 Libraries (296 Components).
- (5) **Scale XL (National ATM Grid):** 156 Apps, 162 Topics, 30 Brokers, 48 Nodes, 48 Libraries (444 Components).

Across scales, QoS mix, fan-out shape, and criticality proportions were held constant, isolating system scale as the sole independent variable. Each scale point was executed across 5 random seeds ($\{42, 123, 456, 789, 2024\}$).

5.2 Verification Cost & CI/CD Overhead

We evaluated end-to-end gate invocation latency across 500 candidate build evaluations per scale:

Table 3: Gate Execution Latency Across 5 ATM Scale Profiles.

Scale Profile	Components	Graph Const. (s)	Rule Audit (s)	Total Gate (s)
Scale Tiny	29	0.008 ± 0.001	0.002 ± 0.000	0.010 ± 0.001
Scale S	74	0.042 ± 0.002	0.008 ± 0.001	0.050 ± 0.003
Scale M	148	0.112 ± 0.005	0.018 ± 0.002	0.130 ± 0.006
Scale L	296	0.485 ± 0.018	0.045 ± 0.004	0.530 ± 0.022
Scale XL	444	0.925 ± 0.031	0.085 ± 0.007	1.010 ± 0.035

Performance Result: Verification latency scales gently with system size. On the **Scale XL National Grid** (444 components, 1,200+ edges), mean audit latency is **1.01 seconds** ($P_{95} < 1.5$ s), demonstrating negligible pipeline overhead.

5.3 Planned Evaluation: Defect Detection Effectiveness in Industrial Pipelines

To measure the empirical effectiveness of SaaG pre-deployment gating once full SaaG-D development concludes, we have designed a 6-month trial protocol across candidate ATM staging pipelines. In this planned trial, all candidate software builds will be passed through the static verification engine prior to staging deployment.

Caught pre-deployment architectural defects will be tracked and categorized across five primary failure classes: (1) Hardware Core Over-Allocation & Conflict on latency-sensitive nodes (`surveillance-server`, `fdps-main`); (2) Middleware Topic QoS Parameter Mismatches on safety channels (`clearance.cmd`, `radar.tracks`); (3) Orphaned Topics & Schema Discord; (4) Circular Software Package Dependencies; and (5) Memory & OS Configuration Incongruity exceeding physical target host bounds.

5.4 Planned Evaluation: Detector Catalog vs. Cascade Propagation Oracle

To evaluate detector precision and scaling behavior on dense architectural graphs, we plan to score the static rule detector catalog against a simulated cascade blast-radius oracle across the 5 ATM scale profiles (Scale Tiny through Scale XL).

For each scale profile, candidate graph instances will be subjected to synthetic failure injection and message burst propagation along structural edges ($E_{\text{structural}}$) to establish ground-truth cascade impact zones. The static rule detectors will be scored using standard binary classification metrics: Precision, Recall, F_1 score, and inter-rater agreement via Cohen's κ coefficient. This study will specifically test the hypothesis of whether static rule-based auditing suffers from structural over-flagging (false positives) as graph density scales up, and measure the degree of precision degradation across system sizes.

5.5 Planned Evaluation: Criticality Ranking Model Comparison

To assess component criticality scoring $Q^*(v)$ on the reference ATM dataset (Scale S), we have structured a comparative benchmark comparing 6 graph representation learning formulations against simulation ground-truth oracles:

- (1) **Topo-BL**: Unweighted Topological Centrality (betweenness and degree).
- (2) **Topo-QoS**: Local QoS-Weighted Centrality.
- (3) **GL**: Homogeneous Graph Attention Network (GAT).
- (4) **GL-QoS**: QoS-Weighted Homogeneous GAT.
- (5) **HGL**: Heterogeneous GAT (QoS Masked).
- (6) **HGL-QoS**: QoS-Aware Heterogeneous GAT (Proposed SaaG Model).

The ranking models will be evaluated using Spearman's rank correlation (ρ), Top- K F_1 score ($K = 20$), Normalized Discounted Cumulative Gain (NDCG@10), and Single Point of Failure (SPOF) identification F_1 score upon completion of full SaaG-D training runs.

5.6 Operational Impact & Threats to Validity

- **Anticipated Operational Impact**: Upon full deployment in target staging pipelines, SaaG gating is designed to reduce post-deployment middleware-related operational incidents and accelerate configuration diagnostic times from hours to seconds by producing evidence-bearing JSON reports.
- **Threats to Validity**: (1) Benchmark detection accuracy will be scored against a cascade simulator, which may not capture all unexpected field hardware dynamics; (2) The generator scales ATM topologies by replicating core application roles, which may differ from real-world sector expansion; (3) SaaG-P implements a subset of SaaG-D's specified capabilities as documented in the gap analysis (§3.3).

6 Related Work

- **Architecture Description Languages (ADLs)**: Languages like AADL and SysML support formal analysis of hardware/software interactions [5]. However, manual model maintenance causes ADLs to drift from implementation code in continuous delivery pipelines. SaaG automates graph model construction per candidate build directly from build artifacts.
- **Static Code Analysis Tools**: Analyzers like SonarQube or Coverity evaluate syntax and local control flows within individual repositories. They cannot observe multi-node topologies, CPU core pinning, or pub/sub QoS contracts across system boundaries. SaaG operates at the architectural system layer.
- **Runtime Application Performance Monitoring (APM)**: Observability frameworks (Prometheus, Dynatrace) track distributed systems post-deployment. While accurate, APMs operate reactively after non-conforming software is deployed. SaaG operates proactively in CI/CD pipelines.
- **Digital Twins in Manufacturing vs. Software**: In Industry 4.0, digital twins synchronize physical assets via live sensor feeds [10]. SaaG adapts the digital twin concept to *software architecture*, scoping its primary verification engine as a static model reconstructed per candidate build.

7 Conclusion & Future Work

We presented **SaaG (System as a Graph)**, an architectural digital twin for pre-deployment verification and continuous deployment gating in mission-critical Air Traffic Management middleware systems. By constructing a multi-attributed multigraph ($G = (V, E)$) from candidate build artifacts, SaaG statically audits hardware core allocations, pub/sub QoS policies (durability, reliability, transport_priority), circular dependencies, and topic integrity prior to installation. Performance benchmarks across 5 ATM scale profiles demonstrated sub-second audit latencies (1.01 s for a 444-component national grid), confirming negligible CI/CD overhead. Furthermore, we documented an industrial gap analysis and established a comprehensive experimental protocol to evaluate defect detection effectiveness, detector precision scaling, and GNN criticality model accuracy during ongoing SaaG-D deployment trials.

Future Work: We plan to complete SaaG-D implementation, execute the planned empirical evaluation trials in live staging pipelines, implement delta-aware gating against merge bases, and integrate

autonomous LLM-driven coding agents to generate remediation pull requests when gating violations occur.

References

- [1] International Civil Aviation Organization (ICAO). 2005. *Global Air Traffic Management Operational Concept*. Doc 9854, AN/458.
- [2] EUROCONTROL. 2020. *EUROCONTROL Specification for ASTERIX Data Category Definitions*, Edition 2.4.
- [3] Object Management Group (OMG). 2015. *Data Distribution Service (DDS) Specification*, Version 1.4.
- [4] L. Bass, P. Clements, and R. Kazman. 2021. *Software Architecture in Practice* (4th ed.). Addison-Wesley Professional.
- [5] P. H. Feiler and D. P. Gluch. 2012. *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language Standard*. Addison-Wesley.
- [6] J. Humble and D. Farley. 2010. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- [7] P. Th. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec. 2003. The many faces of publish/subscribe. *ACM Computing Surveys* 35, 2 (2003), 114–131.
- [8] A. Carzaniga, D. S. Rosenblum, and A. L. Wolf. 2001. Design and evaluation of a wide-area event notification service. *ACM Transactions on Computer Systems* 19, 3 (2001), 332–383.
- [9] U. Brandes. 2001. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology* 25, 2 (2001), 163–177.
- [10] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee. 2019. Digital Twin in Industry: State-of-the-Art. *IEEE Transactions on Industrial Informatics* 15, 4 (2019), 2405–2415.
- [11] R. E. Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM Journal on Computing* 1, 2 (1972), 146–160.
- [12] T. Saaty. 1980. *The Analytic Hierarchy Process*. McGraw-Hill, New York.
- [13] J. Vlissides, R. Helm, R. Johnson, and E. Gamma. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [14] M. Fowler. 2002. *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- [15] E. A. Lee. 2008. Cyber Physical Systems: Design Challenges. In *IEEE ISORC '08*. 363–369.
- [16] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr. 2004. Basic concepts and taxonomy of dependable and secure computing. *IEEE TDSC* 1, 1 (2004), 11–33.
- [17] M. Shaw and D. Garlan. 1996. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall.
- [18] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, and J. Irwin. 1997. Aspect-oriented programming. In *ECOOP '97*. Springer, 220–242.